

Lesson 11 - Page Replacement Algorithms

In an operating system that uses paging, a page replacement algorithm is needed when a page fault occurs and no free page frame is available. In this case, one of the existing pages in memory must be replaced with the new page.

The virtual memory manager performs this by:

- 1. Selecting a victim page using a page replacement algorithm.
- 2. Marking its page table entry as "not present."
- 3. If the page was modified (dirty), writing it back to disk before replacement.

The efficiency of a page replacement algorithm directly affects the page fault rate, which in turn impacts system performance.

Common Page Replacement Techniques

- First In First Out (FIFO)
- Optimal Page replacement
- Least Recently Used (LRU)
- Most Recently Used (MRU)

1. First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Hit - H
Fault - F

Reference String	7	0	1	2	0	3	0	4	2	3	0	3	2	3
Frame 1	7	7	7	7	7	3	3	3	3	3	3	3	3	3
Frame 2	-	0	0	0	0	0	0	4	4	4	4	4	4	4
Frame 3	-	-	1	1	1	1	1	1	1	1	0	0	0	0
Frame 4	-	-	-	2	2	2	2	2	2	2	2	2	2	2
Status	F	F	F	F	H	F	H	F	H	H	F	H	H	H

- Hit = 7
- Fault = 7

2. Least Recently Used

- In this algorithm, page will be replaced which is least recently used.
- For example, if the pages were, 1,2 and 3; and the currently sued page is 2, the one used before was 3 and the one used before 3 was 1; 1 becomes the least recently used page, and becomes replaced.

Hit - H
Fault - F

Reference String	7	0	1	2	0	3	0	4	2	3	0	3	2	3
Frame 1	7	7	7	7	7	3	3	3	3	3	3	3	3	3
Frame 2	-	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3	-	-	1	1	1	1	1	4	4	4	4	4	4	4
Frame 4	-	-	-	2	2	2	2	2	2	2	2	2	2	2
Status	F	F	F	F	H	F	H	F	H	H	H	H	H	H

- Hit = 8
- Fault = 6
- Initially, all frames are empty. When 1, 3, 0 arrive, they are placed in empty frames, resulting in 3 page faults.
- When 3 arrives again, it is already present in memory, so there is no page fault.
- When 5 arrives, it is not in memory, so it replaces the oldest page (1), resulting in 1 page fault.
- When 6 arrives, it is not in memory, so it replaces the oldest page (3), resulting in 1 page fault.
- Finally, when 3 arrives, it is not in memory, so it replaces the oldest page (0), resulting in 1 page fault.

3. Optimal Page Replacement

- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.
- It is The holy grail of page replacement. Instead of looking backward, it looks forward.

- OPR Guarantees the lowest possible number of page faults for any given reference string. Completely immune to Belady's Anomaly.
- However, It is purely theoretical. An operating system cannot accurately predict the future sequence of memory requests. It is primarily used as a benchmark to measure how well other algorithms perform.

Reference String	7	0	1	2	0	3	0	4	2	3	0	3	2	3
Frame 1	7	7	7	7	7	3	3	3	3	3	3	3	3	3
Frame 2	-	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3	-	-	1	1	1	1	1	1	1	1	1	1	1	1
Frame 4	-	-	-	2	2	2	2	2	2	2	2	2	2	2
Status	F	F	F	F	H	F	H	F	H	H	H	H	H	H

- Hit = 8
- Fault = 6
- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ----> 4 Page faults
- 0 is already there so ----> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future----> 1 Page fault.
- 0 is already there so ----> 0 Page fault. 4 will takes place of 1 ----> 1 Page Fault.